5.19-5.24

ollyDbg的使用

代码窗口	默认用于显示反汇编代码，还用于显示各种注释、标签，分析代码时显示循环、跳转位置等信息
寄存器窗口	实时显示CPU寄存器的值，可用于修改特定的寄存器
数据窗口	以Hex/ASCII/Unicode值的形式显示进程的内存地址，也可在此修改内存地址
栈窗口	实时显示ESP寄存器指向的进程栈内存，并允许修改

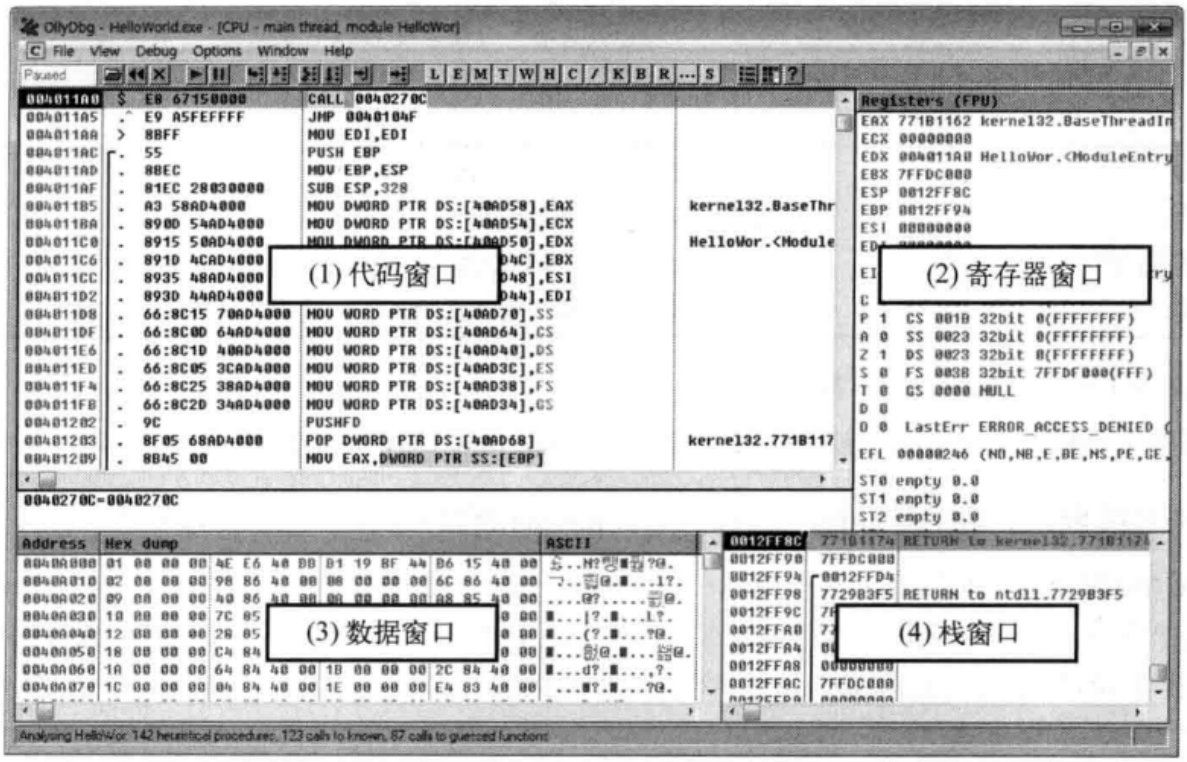

图2-3　OllyDbg基本界面

OllyDbg基本指令（适用于代码窗口）

指　令	快捷键	含　义
Restart	Ctrl+F2	重新开始调试（终止正在调试的进程后再次运行）
Step Into	F7	执行一句OP code（操作码），若遇到调用命令（CALL），将进入函数代码内部
Step Over	F8	执行一句OP code（操作码），若遇到调用命令（CALL），仅执行函数自身，不跟随进入
Execute till Return	Ctrl+F9	一直在函数代码内部运行，直到遇到RETN命令，跳出函数

《论如何保存可执行文件》（又忘了版）

- 保存更改到可执行文件

上面的调试中，我们通过修改字符串缓冲区更改了程序显示的消息内容，但是这种更改只是暂时的，终止调试（即HelloWolrd.exe进程结束）后，程序中的原字符串仍然没有改变。如果想把这种更改永久保存下来，就要把更改后的程序另保存为一个可执行文件。

图2-31的Dump窗口中，选中更改后的“Hello Reversing”字符串，单击鼠标右键，在弹出的菜单中选择Copy to executable file菜单，打开图2-34所示的Hex窗口。

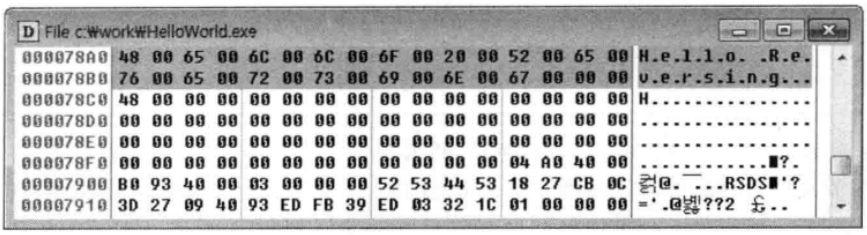

图2-34　Copy to executable file

在弹出的Hex窗口中单击鼠标右键，选择Save file菜单，在Save file as对话框中输入文件名“Hello Reversing.exe”后保存为.exe可执行文件。然后运行该文件，弹出图2-33所示的消息窗口，显示的字符串已经变为“Hello Reversing”。

栈帧

简言之，栈帧就是利用EBP（栈帧指针，请注意不是ESP）寄存器访问栈内局部变量、参数、函数返回地址等的手段。通过前面关于IA-32寄存器的学习我们知道，ESP寄存器承担着栈顶指针的作用，而EBP寄存器则负责行使栈帧指针的职能。程序运行中，ESP寄存器的值随时变化，访问栈中函数的局部变量、参数时，若以ESP值为基准编写程序会十分困难，并且也很难使CPU引用到准确的地址。所以，调用某函数时，先要把用作基准点（函数起始地址）的ESP值保存到EBP，并维持在函数内部。这样，无论ESP的值如何变化，以EBP的值为基准（base）能够安全访问到相关函数的局部变量、参数、返回地址，这就是EBP寄存器作为栈帧指针的作用。

PyInstaller逆向
[thread-2033673-1-1.html]

一些常用的API

访问原数据
idc.get_wide_byte(ea) // 获取单字节，按整形解释
idc.get_wide_word(ea) // 获取双字节，按整形解释
idc.get_wide_dword(ea) // 获取四字节，按整形解释
idc.get_qword(ea) // 获取八字节，按整形解释
idc.GetFloat(ea) //获取四字节，按浮点解释
idc.GetDouble(ea)//获取八字节，按浮点解释
idc.get_bytes(ea,size) // 在ea获取size字节，按byte解释
给数据打补丁
patch_byte(ea, value)//给ea的一个字节，设置值为value
patch_word(ea, value) //给ea的2个字节，设置值为value
patch_dword(ea, value) //给ea的4个字节，设置值为value
patch_qword(ea, value) //给ea的8个字节，设置值为value
获取地址的反汇编
idc.GetDisasm(ea)//获取ea的汇编指令 比如 mov　rbp, rsp
idc.print_insn_mnem(ea)//获取ea的汇编指令的名字，比如mov
idc.print_operand(ea,n)//对立即数的获取，比如rbp，rsp
链接：idapython使用笔记 | wonderkun's | blog

winrar漏洞
17715977.html
s?
__biz=MjM5NTc2MDYxMw==&mid=2458544969&idx=1&sn=473822d99738dc8c20cf0c7df866adea
&poc_token=HFyXMWijd3cHiFN_sKM0IUS4MMCY3YNlizGngCmW

rust
因为 Rust 语言的特性，很难找到具体功能的实现逻辑在哪，调用链有点复杂，实际的加密逻辑与主逻辑包了非常多层，所以采用动调

qemu四种远程调试方法
9662

就这些了
下周放假的时候争取多弄些》……《